



CSC462 – ARTIFICIAL INTELLIGENCE

Lab Finals Report



Submitted By:

Muhammad Rizwan Shafiq
SP24-BCS-069
BCS-4B

Submitted To:

Dr. Muhammad Imran

Table of Contents

Introduction.....	2
Question 1: Genetic Algorithm for Traveling Salesperson Problem (TSP).....	2
Problem Description	2
Genetic Algorithm Components	2
1. Chromosome Representation	2
2. Initial Population.....	2
3. Fitness Function	3
4. Selection Method (Tournament Selection)	3
5. Crossover Operator (PMX).....	3
6. Mutation Operator (Swap Mutation)	4
7. Elitism	4
8. Termination Criteria.....	5
Implementation Details	5
Sample Output	5
Final Result	7
Question 2: Wumpus World Knowledge Base (Prolog)	8
Problem Description	8
World Representation.....	8
Known Facts from the Diagram.....	8
Logical Rules Implemented	8
9. Adjacency	8
10. No-Pit Rule	8
11. No-Wumpus Rule	8
12. Pit and Wumpus Inference	8
Target Queries	9
Sample Output	9
Conclusion	9
References.....	9

Artificial Intelligence Lab Final Project Report

Introduction

This lab final project consists of two independent problems designed to assess practical understanding of **Artificial Intelligence concepts**. The first problem implements a **Genetic Algorithm (GA)** to solve the **Traveling Salesperson Problem (TSP)** using Python. The second problem develops a **knowledge-based agent** for the **Wumpus World** using **Prolog**, based on logical inference from percepts.

The project includes complete source code implementations along with execution screenshots, as required.

Question 1: Genetic Algorithm for Traveling Salesperson Problem (TSP)

Problem Description

The Traveling Salesperson Problem (TSP) requires finding the shortest possible route that visits each city exactly once and returns to the starting city. Since TSP is NP-hard, a **Genetic Algorithm** is used to obtain a near-optimal solution efficiently.

Genetic Algorithm Components

1. Chromosome Representation

Each chromosome represents a **tour**, which is a permutation of city names.

Example:

[A, F, B, I, C, J, G, D, H, E]

2. Initial Population

An initial population of random permutations is generated to ensure diversity.

```
def initial_population(city_names, pop_size):
```

```
    base = city_names[:]
```

```
    population = []
```

```
    for _ in range(pop_size):
```

```
        chrom = base[:]
```

```
        random.shuffle(chrom)
```

```
        population.append(chrom)
```

```
return population
```

3. Fitness Function

The objective is to minimize total tour distance. Fitness is defined as:

Fitness = $1 / (1 + \text{Tour Distance})$

This converts a minimization problem into a maximization problem.

```
def tour_length(tour, dist, name_to_index):
    total = 0.0
    for i in range(len(tour)):
        a = tour[i]
        b = tour[(i + 1) % len(tour)] # return to start
        total += dist[name_to_index[a]][name_to_index[b]]
    return total
```

```
def fitness_from_length(L):
    return 1.0 / (1.0 + L)
```

4. Selection Method (Tournament Selection)

Tournament selection randomly selects k individuals and chooses the one with the minimum tour distance as a parent.

```
def tournament_select(population, lengths, k):
    indices = random.sample(range(len(population)), k)
    best_index = min(indices, key=lambda idx: lengths[idx])
    return population[best_index][:]
```

5. Crossover Operator (PMX)

Partially Mapped Crossover (PMX) is used to ensure valid permutations. It preserves relative city order while preventing duplicates.

```
def pmx_child(parent1, parent2, c1, c2):
    size = len(parent1)
    child = [None] * size

    # 1) Copy middle segment from parent1
    child[c1:c2+1] = parent1[c1:c2+1]

    # 2) Resolve conflicts (mapping)
    for i in range(c1, c2+1):
        if parent2[i] not in child:
            pos = i
```

```

while True:
    value = parent1[pos]
    pos = parent2.index(value)
    if child[pos] is None:
        child[pos] = parent2[i]
        break

# 3) Fill remaining positions from parent2
for i in range(size):
    if child[i] is None:
        child[i] = parent2[i]

return child

def pmx_crossover(parent1, parent2):
    size = len(parent1)
    c1, c2 = sorted(random.sample(range(size), 2))
    child1 = pmx_child(parent1, parent2, c1, c2)
    child2 = pmx_child(parent2, parent1, c1, c2)
    return child1, child2

```

6. Mutation Operator (Swap Mutation)

With a small probability, two cities in a tour are swapped to introduce diversity and avoid premature convergence.

```

def swap_mutation(tour, mutation_rate):
    tour = tour[:]
    if random.random() < mutation_rate:
        i, j = random.sample(range(len(tour)), 2)
        tour[i], tour[j] = tour[j], tour[i]
    return tour

```

7. Elitism

The best `elitism_size` individuals are copied unchanged into the next generation to ensure solution quality never degrades.

```

elite_indices = sorted(range(len(population)), key=lambda idx: lengths[idx])[:elitism_size]
for idx in elite_indices:
    new_population.append(population[idx][:])

```

8. Termination Criteria

The algorithm terminates when:

- A fixed number of generations is reached, or
- No improvement is observed for a given number of generations (patience).

Implementation Details

- Language: Python
- Libraries Used: random, math
- Key Functions:
 - tour_length()
 - fitness_from_length()
 - tournament_select()
 - pmx_crossover()
 - swap_mutation()
 - genetic_algorithm_tsp()

Sample Output

```

Question1_TSP_GA.ipynb ×
Question1_TSP_GA.ipynb > M4 Question 1 – Genetic Algorithm for the Traveling Salesperson Problem (TSP) > M4 Step 6 — Run the Genetic Algorithm >
Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Python 3.13.9

--- INITIALIZATION ---
City order used internally: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
Showing first 3 rows of the distance matrix:
[0.0, 60.8276253029822, 98.99494936611666, 82.46211251235322, 41.23105625617661, 36.05551275463989, 70.7106
[60.8276253029822, 0.0, 60.8276253029822, 80.62257748298549, 76.15773105863909, 31.622776601683793, 41.2310
[98.99494936611666, 60.8276253029822, 0.0, 50.99019513592785, 85.44003745317531, 64.03124237432849, 28.2842

Initial population (showing all tours with distance and fitness):
Chromosome 0: ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'E', 'A', 'B'] | Distance = 494.01, Fitness = 0.002020
Chromosome 1: ['D', 'F', 'C', 'E', 'B', 'I', 'H', 'A', 'G', 'J'] | Distance = 604.79, Fitness = 0.001651
Chromosome 2: ['H', 'F', 'A', 'C', 'E', 'J', 'B', 'G', 'D', 'I'] | Distance = 602.76, Fitness = 0.001656
Chromosome 3: ['H', 'I', 'D', 'A', 'C', 'J', 'B', 'E', 'F', 'G'] | Distance = 597.37, Fitness = 0.001671
Chromosome 4: ['J', 'G', 'H', 'D', 'A', 'C', 'E', 'I', 'F', 'B'] | Distance = 507.50, Fitness = 0.001967
Chromosome 5: ['I', 'A', 'D', 'C', 'H', 'F', 'J', 'E', 'B', 'G'] | Distance = 579.81, Fitness = 0.001722
Chromosome 6: ['F', 'H', 'E', 'J', 'I', 'C', 'G', 'D', 'A', 'B'] | Distance = 509.25, Fitness = 0.001960
Chromosome 7: ['A', 'D', 'J', 'G', 'I', 'C', 'B', 'F', 'H', 'E'] | Distance = 505.19, Fitness = 0.001976
Chromosome 8: ['G', 'A', 'E', 'I', 'F', 'H', 'J', 'D', 'C', 'B'] | Distance = 540.92, Fitness = 0.001845
Chromosome 9: ['E', 'J', 'B', 'H', 'C', 'I', 'G', 'D', 'A', 'F'] | Distance = 575.15, Fitness = 0.001736
Chromosome 10: ['C', 'B', 'A', 'E', 'G', 'H', 'I', 'J', 'F', 'D'] | Distance = 534.81, Fitness = 0.001866
Chromosome 11: ['J', 'A', 'B', 'G', 'I', 'H', 'F', 'E', 'D', 'C'] | Distance = 528.03, Fitness = 0.001890
Chromosome 12: ['D', 'E', 'J', 'B', 'F', 'G', 'A', 'H', 'I', 'C'] | Distance = 541.96, Fitness = 0.001842
Chromosome 13: ['I', 'A', 'F', 'C', 'H', 'E', 'D', 'J', 'B', 'G'] | Distance = 463.87, Fitness = 0.002151
Chromosome 14: ['D', 'H', 'J', 'A', 'C', 'F', 'G', 'E', 'I', 'B'] | Distance = 602.78, Fitness = 0.001656
Chromosome 15: ['D', 'I', 'E', 'B', 'J', 'G', 'F', 'A', 'H', 'C'] | Distance = 519.27, Fitness = 0.001922
Chromosome 16: ['H', 'J', 'F', 'A', 'G', 'C', 'B', 'D', 'I', 'E'] | Distance = 554.90, Fitness = 0.001799
Chromosome 17: ['E', 'F', 'D', 'I', 'J', 'G', 'C', 'B', 'A', 'H'] | Distance = 482.81, Fitness = 0.002067
Chromosome 18: ['A', 'D', 'C', 'F', 'I', 'E', 'G', 'J', 'H', 'B'] | Distance = 580.74, Fitness = 0.001719

Spaces: 4 LF {} Cell 13 of 13

```

```
Best individual in initial population:
Tour: ['I', 'A', 'F', 'C', 'H', 'E', 'D', 'J', 'B', 'G']
Distance: 463.87, Fitness: 0.002151

=== GENERATION 0 ===
Elite individuals carried over:
Elite 13: ['I', 'A', 'F', 'C', 'H', 'E', 'D', 'J', 'B', 'G'] | Distance = 463.87
Elite 17: ['E', 'F', 'D', 'I', 'J', 'G', 'C', 'B', 'A', 'H'] | Distance = 482.81

Example mating in this generation:
Parent 1: ['J', 'A', 'B', 'G', 'I', 'H', 'F', 'E', 'D', 'C']
Parent 2: ['F', 'A', 'J', 'B', 'D', 'G', 'H', 'E', 'C', 'I']
Child 1 (after PMX + mutation): ['F', 'A', 'B', 'G', 'D', 'J', 'H', 'E', 'C', 'I']
Child 2 (after PMX + mutation): ['G', 'A', 'J', 'B', 'I', 'H', 'F', 'E', 'D', 'C']

Summary of this generation:
Best distance: 408.29
Best fitness: 0.002443
Average distance: 503.25
--> New global best found!
```

```
=== GENERATION 1 ===
Elite individuals carried over:
Elite 5: ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E'] | Distance = 408.29
Elite 19: ['D', 'H', 'E', 'B', 'J', 'G', 'F', 'A', 'I', 'C'] | Distance = 412.45

Example mating in this generation:
Parent 1: ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E']
Parent 2: ['E', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'H']
Child 1 (after PMX + mutation): ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E']
Child 2 (after PMX + mutation): ['E', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'H']

Summary of this generation:
Best distance: 408.29
Best fitness: 0.002443
Average distance: 465.34
No improvement in this generation. (No-improve count = 1)
```

```
=== GENERATION 2 ===
Elite individuals carried over:
Elite 0: ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E'] | Distance = 408.29
Elite 2: ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E'] | Distance = 408.29

Example mating in this generation:
Parent 1: ['D', 'H', 'E', 'I', 'J', 'G', 'F', 'A', 'B', 'C']
Parent 2: ['D', 'H', 'E', 'B', 'J', 'G', 'C', 'I', 'A', 'F']
Child 1 (after PMX + mutation): ['D', 'H', 'E', 'B', 'J', 'G', 'F', 'I', 'A', 'C']
Child 2 (after PMX + mutation): ['D', 'H', 'E', 'I', 'J', 'G', 'C', 'A', 'B', 'F']

Summary of this generation:
Best distance: 380.01
Best fitness: 0.002625
Average distance: 432.81
--> New global best found!
```

```
=== GENERATION 3 ===
Elite individuals carried over:
  Elite 8: ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E'] | Distance = 380.01
  Elite 15: ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E'] | Distance = 380.01

Example mating in this generation:
Parent 1: ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E']
Parent 2: ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E']
Child 1 (after PMX + mutation): ['H', 'D', 'C', 'I', 'F', 'G', 'B', 'J', 'A', 'E']
Child 2 (after PMX + mutation): ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E']

Summary of this generation:
  Best distance: 380.01
  Best fitness: 0.002625
  Average distance: 397.39
  No improvement in this generation. (No-improve count = 1)
```

AND SO ON TILL GENERATION 12...

```
Example mating in this generation:
Parent 1: ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E']
Parent 2: ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E']
Child 1 (after PMX + mutation): ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E']
Child 2 (after PMX + mutation): ['H', 'D', 'C', 'I', 'F', 'G', 'J', 'B', 'A', 'E']

Summary of this generation:
  Best distance: 380.01
  Best fitness: 0.002625
  Average distance: 389.31
  No improvement in this generation. (No-improve count = 10)

Stopping early: no improvement for 10 generations.

--- GA FINISHED ---
Best tour found during run:
  H -> D -> C -> I -> F -> G -> J -> B -> A -> E -> H
Best distance: 380.01
Best fitness: 0.002625

FINAL RESULT (returned by function):
  Best tour: H -> D -> C -> I -> F -> G -> J -> B -> A -> E -> H
  Total distance: 380.01
  Fitness: 0.002625
```

Final Result

Best Tour: $H \rightarrow D \rightarrow C \rightarrow I \rightarrow F \rightarrow G \rightarrow J \rightarrow B \rightarrow A \rightarrow E \rightarrow H$

Total Distance: 380.01

Fitness: 0.002625

Question 2: Wumpus World Knowledge Base (Prolog)

Problem Description

This problem models a **Wumpus World** agent using **first-order logic**. Based on percepts such as **stench** and **breeze**, the agent infers the presence or absence of **pits** and the **Wumpus**.

World Representation

- Grid size: 4×4
- Coordinates: (Column, Row)
- Rows decrease downward, columns increase left to right

Known Facts from the Diagram

- Stench: (1,2)
- Breeze: (2,1)
- Visited Cells: (1,1), (2,1)
- **Safe (OK) Cells:** (1,1), (2,1), (1,2), (2,2)

Logical Rules Implemented

Adjacency

Defines 4-directional neighbors (up, down, left, right).

No-Pit Rule

A cell has no pit if:

- It is known safe, OR
- It is adjacent to a safe cell with no breeze

No-Wumpus Rule

A cell has no Wumpus if:

- It is known safe, OR
- It is adjacent to a safe cell with no stench

Pit and Wumpus Inference

- A pit exists if adjacent to a breeze cell and not ruled out
- A Wumpus exists if adjacent to a stench cell and not ruled out

Target Queries

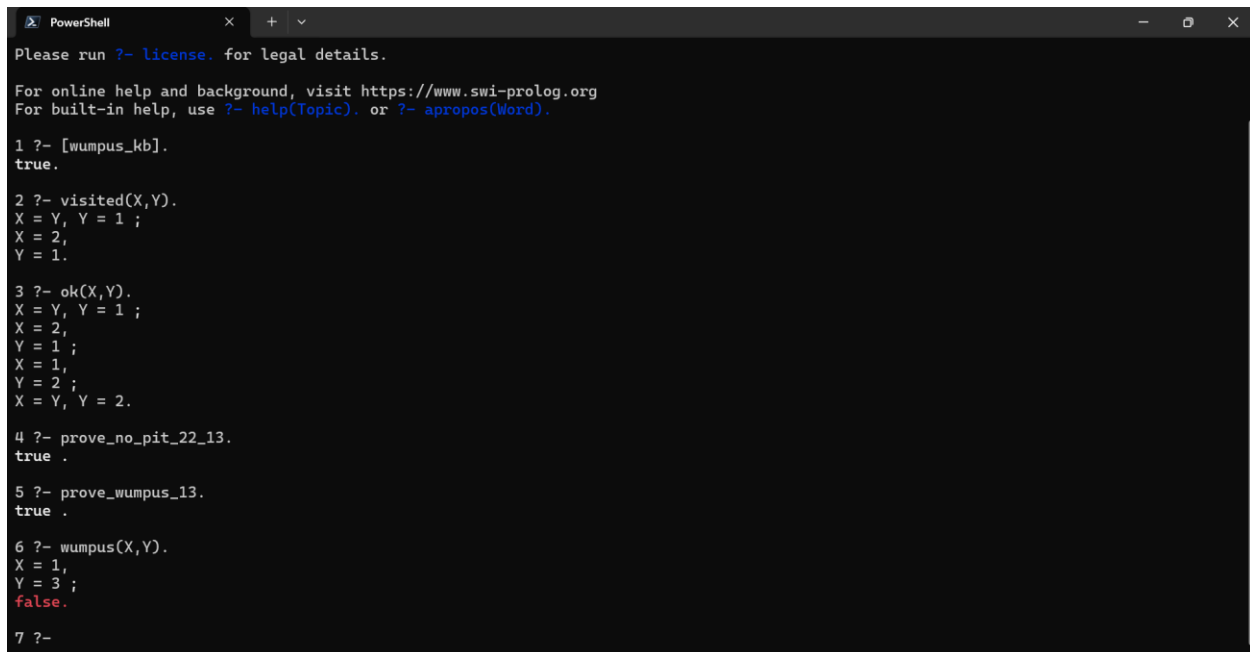
?- prove_no_pit_22_13.

?- prove_wumpus_13.

These queries verify:

- No pit exists at (2,2) and (1,3)
- The Wumpus is located at (1,3)

Sample Output



```
PowerShell
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

1 ?- [wumpus_kb].
true.

2 ?- visited(X,Y).
X = Y, Y = 1 ;
X = 2,
Y = 1.

3 ?- ok(X,Y).
X = Y, Y = 1 ;
X = 2,
Y = 1 ;
X = 1,
Y = 2 ;
X = Y, Y = 2.

4 ?- prove_no_pit_22_13.
true.

5 ?- prove_wumpus_13.
true.

6 ?- wumpus(X,Y).
X = 1,
Y = 3 ;
false.

7 ?-
```

Conclusion

This project demonstrates the application of evolutionary algorithms and logic-based reasoning to solve classical AI problems. The Genetic Algorithm efficiently approximates optimal TSP solutions, while the Prolog knowledge base accurately infers hazards in the Wumpus World using logical rules. Both implementations satisfy the project requirements and illustrate core Artificial Intelligence techniques.

References

- Russell, S., & Norvig, P. (Artificial Intelligence: A Modern Approach)
- SWI-Prolog Documentation
- Python Official Documentation